

AI Security 4.1. Understanding AI Supply Chains

Understanding AI Supply Chains

Task 1 - Introduction

Every time you use Claude, ChatGPT, GitHub Copilot, or any AI-powered product, you are trusting a model trained somewhere, on some data, by someone you have never verified. Every link in that chain is a decision you didn't make, by someone you didn't vet, on infrastructure you don't control.

Imagine you find a model you can download locally that does exactly what you need. The page looks professional: thorough documentation, a credible-sounding organisation name, thousands of downloads. You run `model.load()`. The model works perfectly. What you don't see is that before any prediction ran, it opened a reverse shell to an attacker's server. You now have a stranger with remote access to your system.

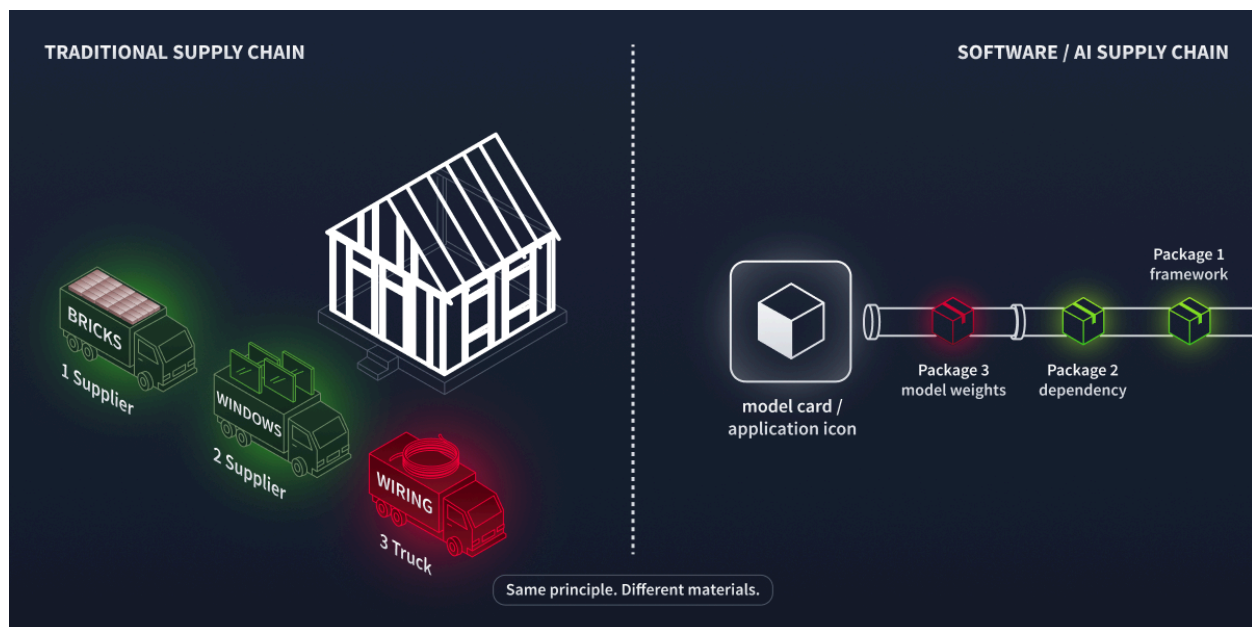
This isn't hypothetical. In 2024, security researchers found **over 100 models** on Hugging Face (the largest public platform for sharing AI models, often called the GitHub of AI) that did exactly this; they were functional, legitimate-looking, and capable of executing arbitrary code the moment they were loaded. This is what makes AI supply chain attacks so effective: **they exploit trust**. You trust model repositories the same way you trust package managers like npm or PyPI. That trust, when misplaced, hands attackers a direct path into your systems. This room introduces the fundamentals of AI supply chains. You will learn what they are, why they differ from traditional software supply chains, and where attackers target them. **Learning Objectives:**

- Explain what an AI supply chain is and how it differs from a traditional software supply chain
- Identify the four key components of an AI supply chain (models, datasets, frameworks, dependencies)

- Map the attack surface across model, dependency, data, and infrastructure layers
- Recognise real-world supply chain incidents and the trust relationships they exploited

Task 2 - What Is a Supply Chain?

Software Supply Chains: Software works the same way. A modern Python application might depend on dozens or sometimes hundreds of third-party packages. When you run `pip install`, you are not just trusting the packages you listed: you are also trusting every **transitive dependency**, a package pulled in automatically by one of your dependencies rather than by you directly. This chain of trust is long, and every link is a potential point of compromise.



Supply chain analogy. A bad supplier component reaches the build site, whether the materials are bricks or bytes.

Why Supply Chain Attacks Work: Supply chain attacks are effective because they **exploit trust rather than bypass defences**. Instead of attacking your application directly, an attacker compromises something your application already trusts.

Consider two high-profile examples from traditional software:

Incident	Year	What Happened	Impact
SolarWinds	2020	Attackers injected malicious code into the Orion build process. Customers installed a legitimate-looking update that contained a backdoor.	~18,000 organisations compromised, including US government agencies
Log4Shell	2021	A critical vulnerability in the widely used Log4j logging library allowed remote code execution.	Affected millions of Java applications worldwide

In both cases, the victims did nothing wrong from their own perspective. They installed trusted software from trusted sources. The compromise happened upstream, in the supply chain itself. The **key insight** here is that supply chain attacks scale efficiently. When an attacker successfully compromises one widely used component, they gain access to every system that depends on it.

Answer the questions below

In the SolarWinds attack, where in the supply chain was the malicious code injected?

Build Process

While installing torch Pip also pulls in filelock, which you never listed. What type of dependency is filelock?

Transitive Dependency

Task 3 - The AI Supply Chain

Traditional software supply chains deal primarily with **code** (including libraries, frameworks, and packages written by developers). AI supply chains add a fundamentally different type of artefact: **trained models**.

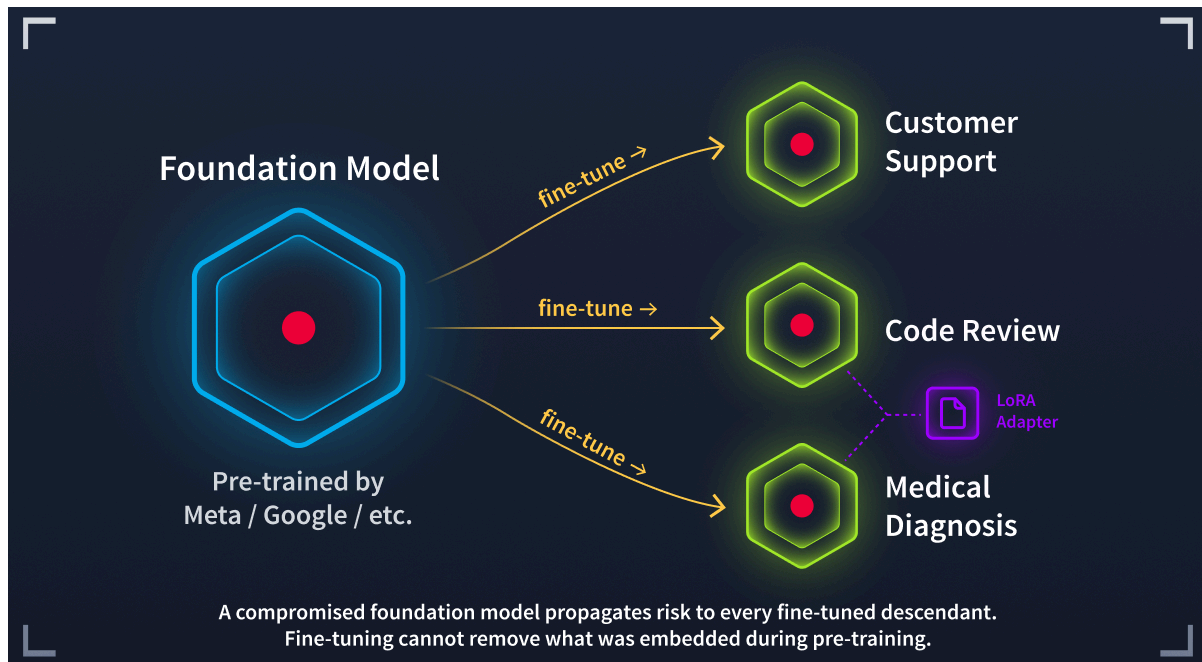
A trained model is not just code. It is the product of an architecture (the neural network structure), training data (potentially millions of examples), a training process (the settings and hardware configuration), and serialised weights (the learned parameters saved to a file). Think of serialised weights like a saved game

state: when you save your progress in a video game, the file captures everything about where you are (your position, inventory, completed quests). Model weights work similarly. They capture everything the model "learned" during training. Now imagine downloading that save file from a stranger. It loads correctly, your character has the right stats, and the game plays as expected. But the person has access to every byte and could have embedded something alongside the legitimate data that stays invisible. Until it matters!.

When you download a pre-trained model, you are trusting all four of these elements, most of which you cannot inspect or verify. Unlike traditional code, which runs as written, pickle-based model files contain **serialised objects** that can execute code the moment you call `model.load()`.

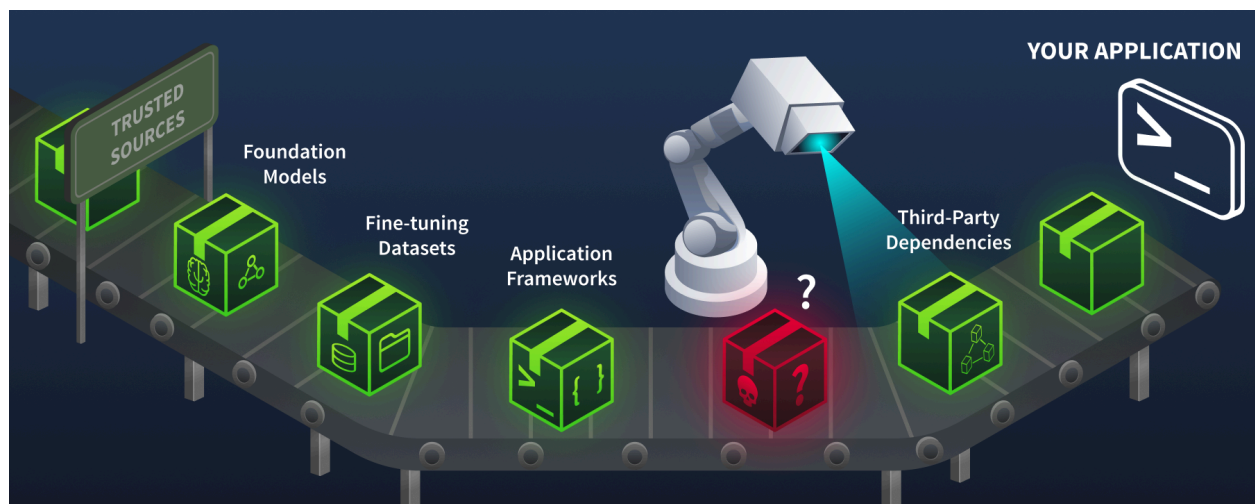
Why This Matters: Transfer Learning: Most teams download a model someone else has already trained and adapt it to their own task, a technique called **transfer learning**. The most common approach is **fine-tuning**: this involves adjusting the model's weights on a smaller, task-specific dataset. A team can fine-tune in hours rather than train from scratch over weeks, but this creates a fundamental security tension: **you are building your application on top of weights trained by an unknown party, on unknown data, using processes you cannot verify**. Research has shown that backdoors inserted during pre-training can **survive fine-tuning**. An attacker who poisons a popular base model does not just compromise one application; they compromise every downstream application that fine-tunes from it.

Modern fine-tuning methods like **LoRA** let teams share small **adapter** files that bolt onto a base model to modify its behaviour. Your supply chain now has two trust dependencies: the base model and the adapter. A clean base model paired with a malicious adapter is still a compromised model.



Fine-tuning adapts the behaviour. It cannot remove what was baked in during pre-training.

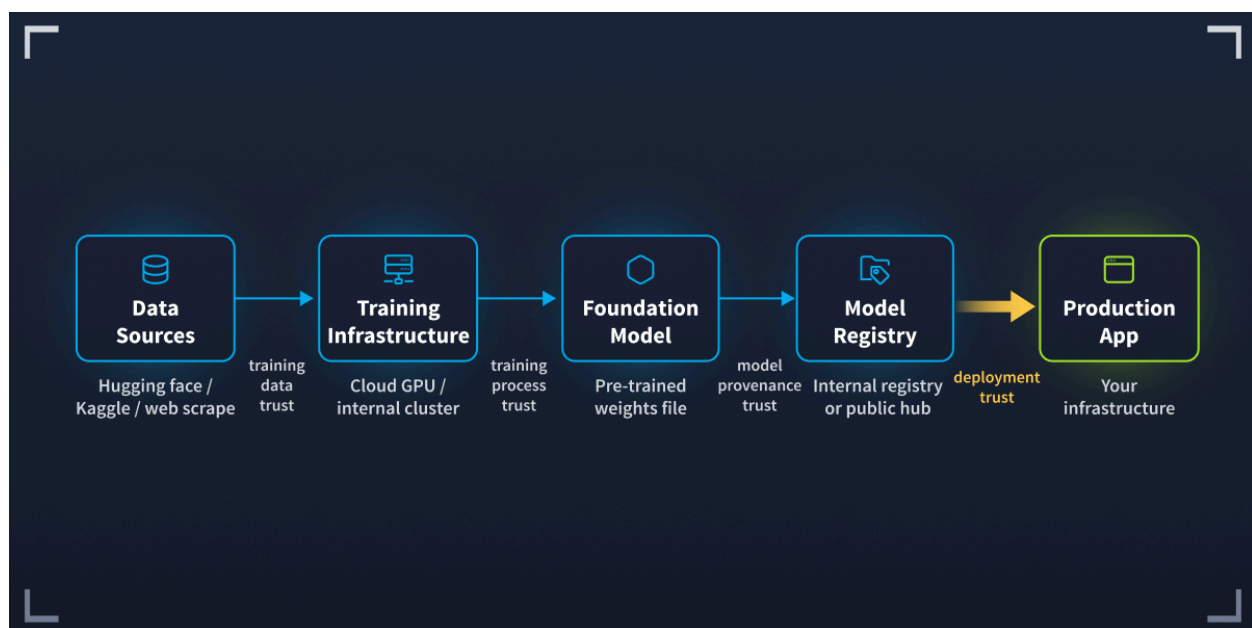
The Four Components: An AI supply chain consists of four key components. Each introduces its own trust relationships and attack surfaces.



Your application trusts the entire supply chain. One malicious component among thousands of legitimate ones is all it takes.

Component	What It Is	Where It Comes From	Example
Models	Pre-trained weights and architectures	Hugging Face Hub, TensorFlow Hub, PyTorch Hub	<code>bert-base-uncased</code> , <code>gpt2</code>
Datasets	Training and evaluation data	Hugging Face Datasets, Kaggle, academic repositories	ImageNet, Common Crawl
Frameworks	ML libraries that train and run models	PyPI, conda, GitHub	PyTorch, TensorFlow, scikit-learn
Dependencies	Supporting packages the frameworks rely on	PyPI, npm, conda-forge	NumPy, SciPy, Pillow, tokenizers

The Architecture: Here is how these components fit together in a typical ML deployment:



Every arrow connects a component you depend on but did not build.

Every arrow in this diagram represents a **trust relationship**. Your application trusts the framework. The framework trusts the package registry. The registry trusts whoever uploaded the package. If any of these trust relationships is broken, the entire system is compromised.

Answer the questions below

What are the four key components of an AI supply chain?

datasets, dependencies, frameworks, models

What do model files contain that allows them to run code when loaded?

Serialised Objects

Task 4 - Two Ways to Consume AI

Not every AI supply chain looks the same. There are two fundamentally different ways organisations consume AI models, and each creates a different risk profile.

Paradigm 1: Downloading Model Files: This is the traditional approach, where you download pre-trained weights (`.pkl` , `.safetensors` , `.pt` , `.gguf` , etc.) from a repository like Hugging Face and load them into your own infrastructure. You are responsible for hosting, inference, and security.

In this paradigm, everything you load crosses your trust boundary and runs on your own systems. The file format determines which risks are in play. `.pkl` , `.pt` , and `.bin` files use Python's pickle serialisation, which can execute arbitrary code the moment a file is loaded. `.safetensors` eliminates that risk by storing only raw weight values with no executable code. `.h5` is the native format for Keras, a popular deep learning framework: not pickle-based, but it can contain executable architecture-level code embedded in the model's layers. `.gguf` , the dominant format for running local large language models such as LLaMA, Mistral, and Qwen, is not pickle-based and does not carry a serialisation exploit. Weight-level attacks and backdoors still apply to GGUF files, however.

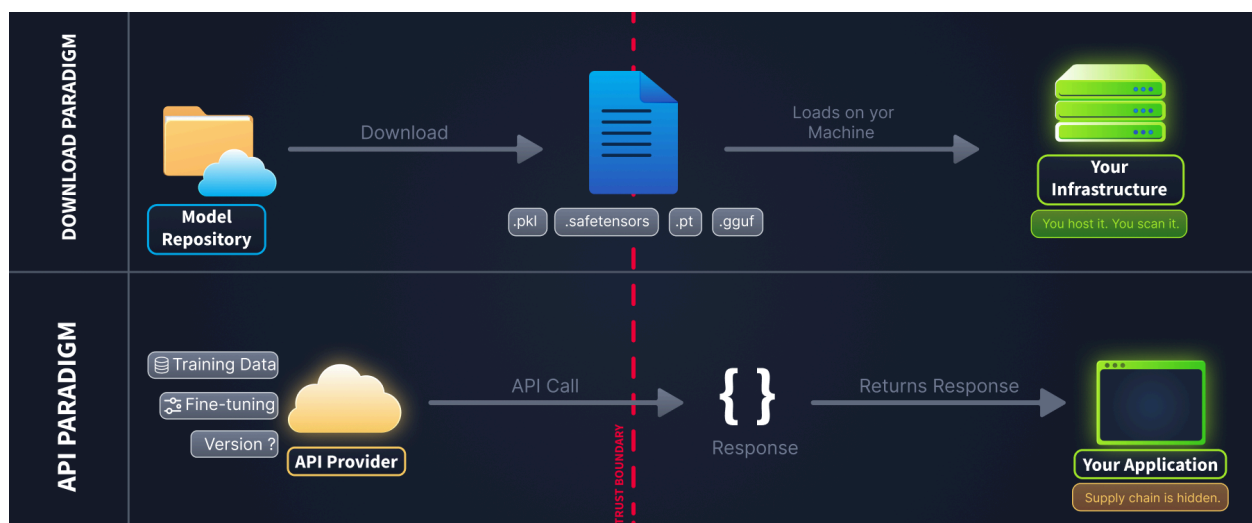
GGUF models are almost always quantised. Quantisation compresses a model's weights from full precision (32-bit floating-point) to lower precision (4- or 8-bit integers), reducing file size so large models can run on consumer hardware. If a third party performed that compression, the quantisation step is itself an unverified point in the supply chain: you are trusting not just the original training but every transformation the file went through before reaching you.

Paradigm 2: Calling Models via API: Increasingly, organisations consume AI through **hosted API services** from providers such as OpenAI,

Anthropic, and Google, or aggregators such as OpenRouter. You send a prompt, the provider runs inference, and you receive a response. You never touch the model file. This might seem safer (no pickle files to worry about), but you are still trusting a supply chain. It is just a **different** supply chain:

Supply Chain Element	Download Paradigm	API Paradigm
Model weights	You inspect/scan them	Provider controls them; you cannot inspect
Training data	Often documented in a model card	Often undisclosed or vaguely described
Fine-tuning	You control it	Provider may fine-tune without notice
System prompts	Not applicable	Templates from untrusted sources can alter behaviour
Versioning	You pin a specific file hash	Provider may update the model behind the same API endpoint
Hosting	Your infrastructure	Provider's infrastructure, shared, multi-tenant

Key insight: When you call a model through an API, you are trusting the provider's entire pipeline: their training data curation, fine-tuning decisions, hosting security, and update practices. You cannot verify any of these independently. The supply chain is still there; it is simply hidden behind an API call.



Download paradigm: a model file (.pkl, .safetensors, .pt, .gguf) crosses your trust boundary and runs on your infrastructure. API paradigm: only a JSON response crosses your boundary, but the supply chain behind it is invisible.

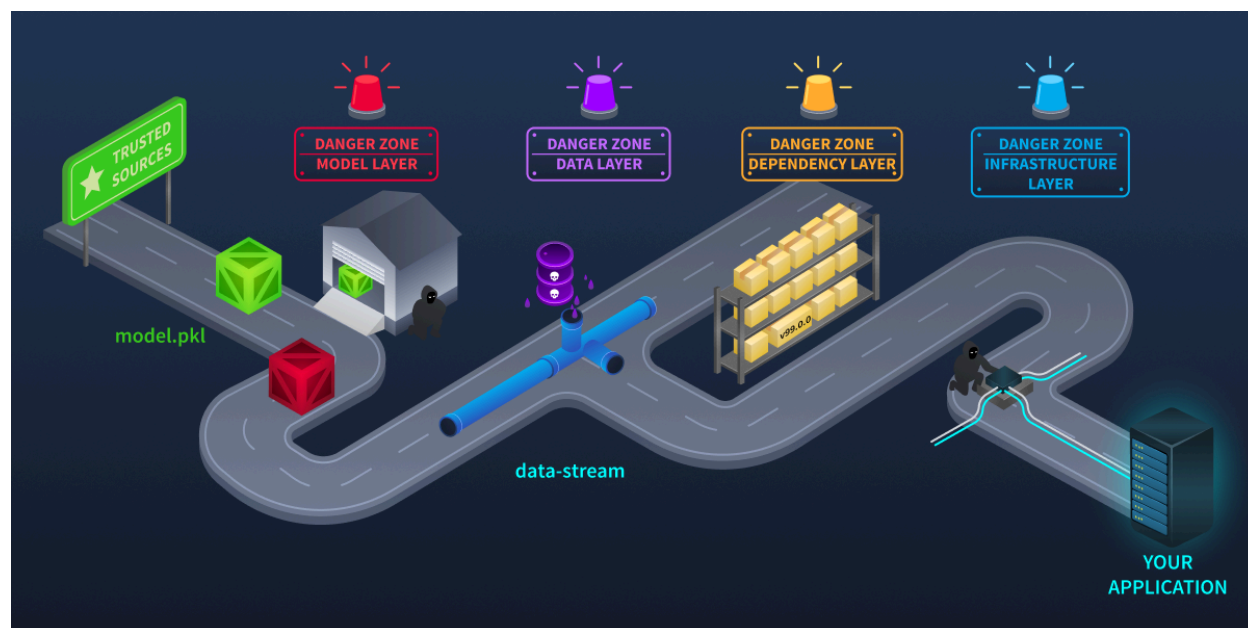
Answer the questions below

What is the dominant file format for running local large language models such as LLaMA, Mistral, and Qwen?

gguf

Task 5 - The Four Attack Layers

In this task, we will map out the **attack surface** for the download paradigm: the four distinct layers where attackers can strike. Understanding these layers will help you know what to look for when evaluating models, packages, and data sources.



Every stop on the supply chain route is a potential interception point. A sophisticated attacker does not need to own the entire chain. Just one checkpoint.

Layer 1: The Model Layer

The model layer is the most distinctive part of the AI attack surface. Attackers embed executable code inside model files that runs silently at load time, modify a model's architecture to trigger on specific inputs, or subtly alter trained weights to introduce hidden behaviours.

Model Layer Attack Taxonomy: The model layer is the most distinctive part of the AI attack surface. Attackers embed executable code inside model files that runs silently at load time, modify a model's architecture to trigger on specific inputs, or subtly alter trained weights to introduce hidden behaviours. Not all model-layer attacks work the same way. Three distinct levels have been identified, and each behaves differently:

- **Serialisation-level:** Code hidden inside the file format itself, and executes when the file is loaded
- **Architecture-level:** Malicious logic embedded in the model's layers, executed on every prediction
- **Weights-level:** Learned values subtly altered to misbehave on specific inputs

Key insight: Stripping executable code from a model file eliminates serialisation-level attacks but leaves architecture-level and weights-level attacks completely untouched. Effective defence requires inspection at every level. The [Securing the AI Supply Chain](#) room covers the tools for each of them.

Layer 2: The Dependency Layer

This layer covers the packages and libraries your ML project depends on. Attackers exploit it through dependency confusion, typosquatting, and uploading malicious packages with professional-looking descriptions that hide malware.

Layer 3: The Data Layer

Training data is the foundation of every ML model, and poisoning it is difficult to detect. [Researchers have demonstrated](#)(opens in new tab) that replacing as little as 0.1% of a training dataset with crafted samples can introduce a reliable backdoor without measurably affecting accuracy on clean data. For denial-of-service objectives, the effective threshold drops as low as 0.001%.

Note: Data poisoning attacks are covered in depth in the [Data Poisoning](#) module. This room focuses on the supply chain context: how

poisoned data reaches your pipeline through untrusted sources.

Layer 4: The Infrastructure Layer

The infrastructure layer covers the systems that host, distribute, and manage AI artefacts. Attackers gain access to model or package repositories, inject malicious steps into CI/CD build pipelines, or steal maintainer credentials to push malicious updates under trusted identities.

The Compounding Effect: These layers do not exist in isolation. A sophisticated attacker can combine techniques across layers. For instance, an attacker could:

- **Compromise an account** on Hugging Face (infrastructure layer)
- **Upload a backdoored model** with malicious pickle code (model layer)
- **Include a requirements.txt** referencing typosquatted packages (dependency layer)
- **Distribute a poisoned dataset** alongside the model (data layer)

A single download can trigger compromises at every layer simultaneously.

Answer the questions below

At which layer of the AI supply chain do pickle-based attacks occur?

Model Layer

Which level of model attack is eliminated by converting to SafeTensors format?

Serialisation-level

Researchers find that 0.1% of a public training dataset has been replaced with crafted samples designed to introduce a backdoor. Which attack layer does this represent?

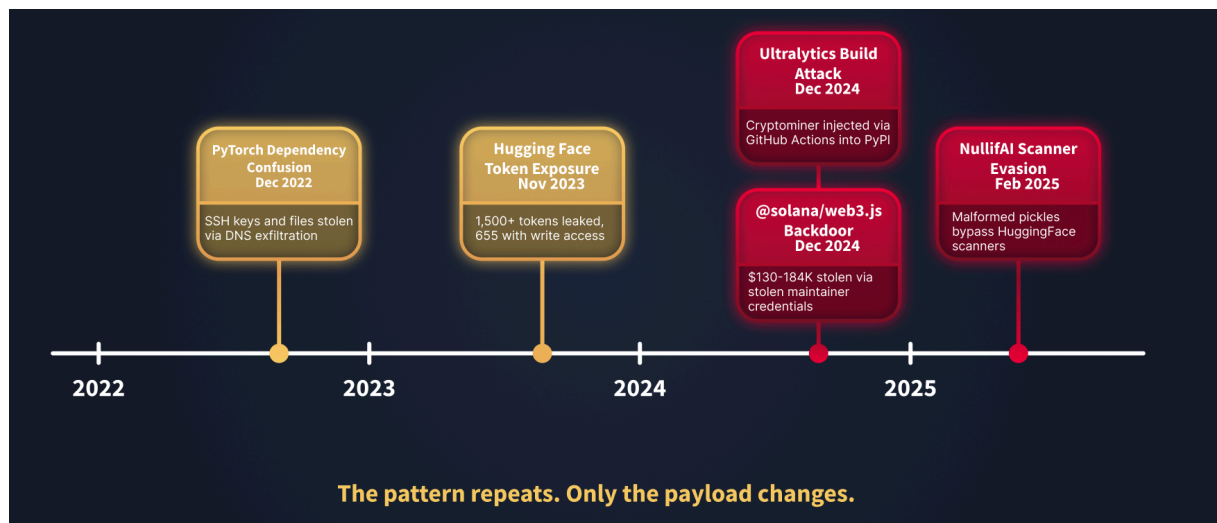
Data Layer

Task 6 - Real-World Incidents

The attack techniques from the previous task are not theoretical; they have all been used in real attacks. In this task, we will examine five significant incidents

from 2022 to 2025. Each one exploited a different part of the supply chain, and together they show the full range of risks you need to defend against.

The deception at the heart of supply chain attacks: A model that looks trustworthy on the surface can hide malicious code inside. Don't judge a model by its star rating.



Incident 1: PyTorch Dependency Confusion (December 2022): On Christmas Day 2022, an attacker published a malicious package named `torchtriton` on PyPI, exploiting the fact that PyTorch's nightly builds depended on an internal package with the same name. pip's version resolution installed the attacker's public version instead. The package stole SSH keys, Git configuration, `/etc/passwd`, and up to 1,000 home directory files, exfiltrating everything via encrypted DNS queries. It persisted for five days before discovery. Reference: [PyTorch blog \(opens in new tab\)](#). "Compromised nightly dependency" (December 2022)

Incident 2: Hugging Face Hub Vulnerabilities (2023–2024): Multiple security issues emerged on the world's largest ML model platform. In November 2023, Lasso Security found over 1,500 exposed API tokens, 655 of which had write permissions, allowing malicious actors to push updates to legitimate repositories. In 2024, JFrog identified approximately 100 malicious pickle-based models that looked legitimate, with professional descriptions and real ML functionality, while silently executing attacker code at load time. The [Supply Chain Attack Vectors](#) room examines exactly how. References:

- [Lasso Security\(opens in new tab\)](#). "1,500 HuggingFace API tokens were exposed" (November 2023)
- [JFrog Security Research\(opens in new tab\)](#). "Data scientists targeted by malicious Hugging Face ML models" (2024)

Incident 3: Ultralytics Build Pipeline Compromise (December 2024): Attackers injected malicious code into the GitHub Actions build workflow for Ultralytics, the organisation behind the widely used YOLO object detection library. The compromise caused a cryptominer to be embedded into published PyPI packages, meaning developers who ran a standard `pip install` received the malicious version. The attack targeted the built infrastructure rather than the source code directly. Reference: [PyPI blog\(opens in new tab\)](#). "Ultralytics attack analysis" (December 2024)

Incident 4: @solana/web3.js Compromise (December 2024): Attackers stole a maintainer's npm credentials and published two backdoored versions of a widely-used package, exfiltrating cryptocurrency private keys from wallet holders. Estimated losses reached **\$130,000–\$184,000** in the hours before detection. Though not AI-specific, the pattern is identical to what AI supply chains face: trusted identity, upstream compromise, victims who did nothing wrong. Reference: [Anza\(opens in new tab\)](#). "web3.js exploit root cause analysis" (December 2024)

Incident 5: NullifAI Scanner Evasion on Hugging Face (February 2025): ReversingLabs researchers discovered that deliberately corrupted pickle files could bypass Hugging Face's automated security scanners entirely while still executing malicious code when loaded by a developer. The technique exploited edge cases in how scanners parse malformed pickle structures, meaning a model could pass all platform-level checks and still be dangerous. Reference: [ReversingLabs\(opens in new tab\)](#). "RL identifies malware ML model hosted on Hugging Face" (February 2025)

Answer the questions below

The torchtriton package exploited pip's version resolution to install a public package over an internal one. Which of the four attack layers does this target?

Dependency Layer

The @solana/web3.js attacker stole a maintainer's credentials to push malicious updates to a legitimate, high-trust repository. Which attack layer does this represent?

Infrastructure Layer

Task 7 - Practical

Now it is your turn to practise evaluating models the way you will encounter them in real life: **on repository pages**. Click the **View Site** button below. You will see a realistic Hugging Face model page, a similar interface to what you would use when browsing for models.

Your scenario: You are a developer at a startup. Your team lead found a sentiment analysis model (`trustworthy-ai-models/bert-sentiment-classifier`) and wants to deploy it to production. Before you run `pip install` and `model.load()` , you need to evaluate whether it is safe to do so.

The page opens on the **Model Under Review**: the one your team lead wants to use. Explore it as you would any model repository:

1. **Model Card tab:** Read the documentation. Note whether it is thorough or sparse.
2. **Files tab:** Examine the model weight format. Check for any warnings.
3. **Security tab:** Hugging Face now shows automated security scan results. Review what the scanner found.
4. **Sidebar:** Check the organisation, download count, and creation date.
5. **Community tab:** Check for discussions or warnings from other users.

Then click "**Compare: Verified Model**" to see what a legitimate, trustworthy model looks like. This is `google-bert/bert-base-uncased` , one of the most widely used models on Hugging Face, from a verified organisation with a 6+ year track record.

Compare these signals between the two models:

Signal	Suspicious Model	Legitimate Model (baseline)
Organisation	Verified? When did this account join?	Verified, established track record

Signal	Suspicious Model	Legitimate Model (baseline)
Downloads	How many last month?	Millions of downloads
File format	Pickle or SafeTensors?	SafeTensors
Security scan	Any issues flagged?	No issues found
Model card	Thorough or sparse?	Thorough, with training data and limitations documented

After exploring both, answer the questions below. Then ask yourself: **would you approve the first model for production?**

Answer the questions below

In the static site, what is the name of the unverified organisation that uploaded the model?

trustworthy-ai-models

The screenshot shows the Hugging Face interface for the 'bert-sentiment-classifier' model. The header includes the Hugging Face logo and navigation links: Models, Datasets, Spaces, Docs, Pricing. A warning banner reads: 'Exercise: Should You Trust This Model? Your team lead found a sentiment analysis model and wants to deploy it to production. Evaluate it, then compare it against a known-good model. V'. Below this are two buttons: 'Model Under Review' (red dot) and 'Compare: Verified Model' (green dot with checkmark). The breadcrumb trail is 'trustworthy-ai-models / bert-sentiment-classifier'. The model name 'bert-sentiment-classifier' is prominently displayed. Below it are tags: Text Classification, Transformers, PyTorch, Pickle Format, English, sentiment. A navigation bar shows 'Model card' (selected), Files, Security, and Community (2). The main content area is titled 'BERT Sentiment Classifier' and describes it as 'A fine-tuned BERT model for sentiment analysis. Achieves state-of-the-art results on standard benchmarks. Easy to use with the Transformers library.' Under 'How to Use', there is a code block with the following Python code:

```
from transformers import pipeline

classifier = pipeline("sentiment-analysis", model="trustworthy-ai-models/bert-sentiment-classifier")
result = classifier("I love this product!")
print(result)
```

How many downloads does this model have (last month)? 127

20

Follow

benchmarks. Easy to use with the Transformers

```
rt-sentiment-classifier")
```

fy text into positive, negative, or neutral

MODEL STATS

Downloads (last month)	127
Likes	23
Created	Jan 28, 2025

ORGANIZATION

T

trustworthy-ai-models

Not verified

Joined January 2025 · 1 model

SECURITY NOTICE

This model uses **pickle serialization** which can execute arbitrary code. Consider models using **SafeTensors** format instead.

What file format does the verified model (google-bert/bert-base-uncased) use for its weights?

SafeTensors


[Models](#)
[Datasets](#)
[Spaces](#)
[Docs](#)
[Pricing](#)


Exercise: Should You Trust This Model?
 Your team lead found a sentiment analysis model and wants to deploy it to production. Evaluate it, then compare it against a known-good model.

Model Under Review
Compare: Verified Model


 google-bert / bert-base-uncased

bert-base-uncased
Verified

Fill-Mask
Transformers
PyTorch
SafeTensors
English
bert

[Model card](#)
[Files](#)
[Security](#)
[Community](#)
91

 README.md	13.1 KB
 config.json	570 B
 model.safetensors safetensors	440 MB
 tokenizer.json	466 KB
 vocab.txt	232 KB

Task 8 - Conclusion

The attacks in this room share a common thread. The `torchtriton` package stole keys from developers who ran a routine installation command. The malicious models on Hugging Face performed their advertised ML task correctly while silently connecting to servers controlled by attackers. The Solana package compromise lasted only hours but caused six-figure losses. **None of these attacks required breaking into anything.** In each case, the victim trusted a component that had earned that trust, and that trust was the vulnerability.

That is what makes supply chain attacks particularly hard to defend against. The malicious models JFrog found on Hugging Face were not obviously wrong. They had professional model cards, plausible organisation names, and real functionality (they performed their advertised task correctly). They passed every informal

check an ML engineer would reasonably run. The compromise was invisible until anomalous network connections surfaced, sometimes weeks later.

Key takeaways from this room:

- **Model files are code in disguise.** Pickle-based models execute at load time; format choice determines your exposure.
- **The four attack layers are distinct.** Model, dependency, data, and infrastructure attacks require different defences and produce different forensic signals.
- **Transitive dependencies silently expand your attack surface.** You are responsible for every package pip installs, not just the ones you listed.
- **Automated scanners are not a complete defence.** The NullifAI case shows that malformed files can pass platform checks while remaining executable.
- **Infrastructure compromise beats content filtering.** Stolen credentials on a trusted repository push malicious code with full legitimacy; no tooling flags it at download time.
- **SafeTensors eliminates serialisation-level attacks.** It does not eliminate architecture-level or weight-level attacks; the format is a single control, not a complete solution.